

课程笔记

CS146S Week 6: AI Testing and Security

基于 Mihail Eric + Isaac Evans (Sengrep CEO) 授课内容整理

2025 年 10 月

课程: Stanford CS146S

学期: Fall 2025

讲次: 1 lecture (Mon)

网站: <https://themodernsoftware.dev>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 软件安全的基础知识	2
1.1 为什么安全比以往更重要	2
1.2 传统威胁图景	2
1.3 本章小结	2
2 三大漏洞检测技术	2
2.1 SAST: 静态应用安全测试	2
2.2 DAST: 动态应用安全测试	3
2.3 SCA: 软件组成分析	3
2.4 本章小结	3
3 AI Agent 带来的新型攻击向量	3
3.1 Prompt Injection	3
3.2 Tool Misuse	3
3.3 Intent Breaking	3
3.4 Identity Spoofing	4
3.5 Code Attacks	4
3.6 本章小结	4
4 AI 在安全测试中的应用与局限	4
4.1 AI 改善安全的方式	4
4.2 当前的严峻局限	4
4.3 开放问题	4
4.4 把 AI 安全检查嵌入开发流程	5
4.5 本章小结	5
5 总结与延伸	5
5.1 关键点	5
5.2 团队落地建议	6
5.3 事故响应视角下的 AI 安全	6
5.4 安全策略总览表	6
5.5 拓展阅读	6

1 软件安全的基础知识

1.1 为什么安全比以往更重要

软件错误可以摧毁用户信任并带来巨大的财务损失。当 LLM 编写了你的大部分代码时，你需要更加广泛的防护栏来防止这些错误。

AI 编码时代的安全悖论

AI 编码工具提高了代码产出速度，但同时也增加了引入安全漏洞的风险。代码审查的人类工程师成为最后的安全防线——审查一段你没亲手写的代码比审查自己写的代码更具挑战性。

1.2 传统威胁图景

常见的软件安全威胁包括：

- **SQL Injection**：通过恶意 SQL 输入操纵数据库
- **Cross-Site Scripting (XSS)**：在网页中注入恶意脚本
- **Broken Authentication**：认证机制的缺陷
- **Insecure Direct Object References (IDOR)**：直接对象引用未经授权检查
- **Security Misconfigurations**：安全配置错误
- **Sensitive Data Exposure**：敏感数据泄露

1.3 本章小结

传统安全威胁在 AI 编码时代并未消失，反而因代码产出速度的提升而更加需要系统性的防护机制。

2 三大漏洞检测技术

2.1 SAST：静态应用安全测试

- 白盒测试技术，分析源代码和二进制文件
- 在 SDLC 早期执行，此时发现和修复的成本最低
- 可识别：SQL injection、command injection、XSS
- 核心技术：基于模式匹配的代码库扫描

“Shift Left” 安全理念

SAST 体现了“shift left”安全理念——将安全检查尽可能前移到开发生命周期的早期。在代码编写阶段就发现漏洞，修复成本远低于在生产环境中发现后再修复。AI 让这一理念比以往更容易实践。

2.2 DAST: 动态应用安全测试

- 黑盒测试技术，模拟真实攻击者的行为
- 可贯穿整个 SDLC，假阳性率较低
- 可识别：SQL injection、broken authentication、XSS
- 核心技术：
 - Input fuzzing（输入模糊测试）
 - Session token manipulation（会话令牌操纵）
 - Configuration/header testing（配置/头部测试）
 - Brute force rate-limit tests（暴力破解速率限制测试）

2.3 SCA: 软件组成分析

- 深度分析应用使用的 OSS 包
- 分析包管理器、infrastructure-as-code、拉取的镜像
- 核心技术：
 - 分析包元数据的依赖关系
 - 传递依赖解析（transitive dependency resolution）
 - 匹配已知漏洞数据库
 - 二进制/artifact 扫描

2.4 本章小结

SAST、DAST、SCA 三种技术互补：SAST 在早期通过代码分析发现漏洞，DAST 在运行时模拟攻击，SCA 分析第三方依赖的安全性。完整的安全策略需要三者结合。

3 AI Agent 带来的新型攻击向量

3.1 Prompt Injection

在 AI 系统中隐藏或注入误导性指令，使其偏离预期行为。这是 AI Agent 面临的最核心安全威胁。

3.2 Tool Misuse

通过欺骗性 prompt 操纵 Agent 滥用其集成的工具。例如，诱导 Agent 执行未授权的文件操作或 API 调用。

3.3 Intent Breaking

操纵 Agent 的执行计划（plan），将其行动从原始意图重定向到攻击者的目标。

3.4 Identity Spoofing

利用被攻破的认证机制冒充合法 Agent。

3.5 Code Attacks

利用 Agent 执行代码的能力，获取对执行环境的未授权访问。

AI Agent 的攻击面远大于传统软件

传统软件的攻击面主要在输入验证和认证环节。AI Agent 的攻击面则扩展到了 prompt（可被注入）、工具调用（可被滥用）、执行计划（可被篡改）、身份认证（可被伪造）和代码执行环境（可被利用）等多个维度。

3.6 本章小结

AI Agent 引入了至少五种新型攻击向量。防护需要在 prompt 过滤、工具权限控制、计划验证、身份验证和沙箱隔离等多个层面同时展开。

4 AI 在安全测试中的应用与局限

4.1 AI 改善安全的方式

- “Shift left” 安全比以往更易实现
- LLM 可以在工作流中发现问题
- 自动化渗透测试 (automated penetration testing)

4.2 当前的严峻局限

AI SAST 的假阳性率问题

在 AI SAST 中，假阳性率极高：Claude Code/Codex 的假阳性率可达 50–100%（取决于漏洞类型），而传统 SAST 技术的假阳性率也在 50%+ 以上。这意味着大量的安全告警是误报，会严重干扰开发者的工作流。

- **基准不现实**：现有 benchmark 往往不反映真实场景，难以评估 LLM 的安全分析能力
- **非确定性分析**：同一 prompt 运行多次得到不同结果——如何确保捕获所有漏洞？
- **Context rot**：并非所有 context 都同等有效
- **Compaction 信息丢失**：为适应 context window 而进行的摘要压缩可能丢失关键安全细节

4.3 开放问题

1. 如何降低漏洞检测中的假阳性和 hallucination？
2. 如何验证 LLM 生成的补丁是安全的且不引入回归？

- 3. LLM 能否解释**为什么**标记某个漏洞或提出某个修复方案？
- 4. 什么是衡量 LLM AppSec 性能的正确 benchmark？
- 5. LLM 应该如何嵌入 CI/CD 流程而不会用噪音淹没团队？
- 6. 如果 AI 生成的补丁引入了漏洞，谁应该负责？

4.4 把 AI 安全检查嵌入开发流程

课程内容落到工程实践时，最关键的不是再加一个“智能扫描器”，而是把它放到合适的位置：

阶段	适合的 AI 安全动作	主要目标
编码阶段	IDE 内联提示、局部静态检查	尽早阻止明显的危险模式进入代码库
提交阶段	PR 级漏洞复核、依赖变更扫描	在合并前拦截高风险改动
测试阶段	自动化 DAST / fuzzing 辅助	发现运行时路径和环境相关问题
上线后	监控告警摘要、事故复盘辅助	缩短发现与定位时间，形成经验闭环

表 1: AI 安全能力在开发流程中的嵌入点

最有效的策略通常不是“让 AI 独立负责安全”

更现实的做法是把 AI 放在“高召回、低决策权”的位置：它负责尽可能早地暴露风险，人类和硬性测试系统负责最终裁决。这样既能利用 AI 的覆盖面，又能控制假阳性和幻觉带来的噪音成本。

4.5 本章小结

AI 在安全测试领域既是机遇也是挑战。LLM 可以降低“shift left”的门槛，但假阳性率高、非确定性输出和 context 管理问题仍需解决。责任归属是一个尚未回答的关键伦理问题。

5 总结与延伸

Week 6 聚焦 AI 时代的软件测试与安全，呈现了一幅“双刃剑”图景：AI 既提升了安全测试的效率和覆盖面，也引入了全新的攻击向量。

5.1 关键点

- SAST（白盒）、DAST（黑盒）、SCA（依赖分析）三种技术互补
- AI Agent 引入五种新型攻击向量：prompt injection、tool misuse、intent breaking、identity spoofing、code attacks
- AI SAST 假阳性率 50-100%，是当前最大的实用性障碍

- 非确定性输出使“完整覆盖”成为开放问题
- “Shift left”安全理念在 AI 时代更易实践
- AI 生成代码的安全责任归属是未回答的伦理问题

5.2 团队落地建议

1. 先把传统安全基线做好，再引入 AI 辅助检查，避免把 AI 当成“缺失安全工程”的替代品
2. 为 AI 安全告警设置分级和升级路径，防止高假阳性直接压垮团队注意力
3. 对 AI 生成的安全补丁保留更严格的 review 和测试门槛，特别是认证、权限和支付链路

5.3 事故响应视角下的 AI 安全

Week 6 还有一个很实用的启发：安全不只是预防，更是响应。对于已经接入 Agent 的团队，事故处理流程也需要调整：

- 先确认问题来自传统漏洞、提示注入还是工具滥用
- 保留 Agent 的执行轨迹、prompt、工具调用日志和环境状态
- 区分“模型误判”与“系统权限设计错误”，两者的修复路径完全不同
- 把事故复盘重新写回规则、测试和权限边界，而不是只修一个表面 bug

如果没有日志，很多 AI 安全事故几乎不可复盘

传统 Web 服务至少有请求日志、错误堆栈和数据库审计；但 Agent 系统如果不记录 prompt、工具输入输出和关键中间决策，事后往往只能看到错误结果，却不知道它为什么做出这个动作。这会让修复停留在猜测层面。

5.4 安全策略总览表

风险面	主要防护手段	课程给出的原则
传统代码漏洞	SAST / DAST / SCA 组合	不依赖单点检测，尽量前移到开发早期
Agent 提示攻击	Prompt 过滤、权限隔离、计划验证	把 prompt 当成新的攻击输入面来设计防护
工具滥用与身份冒充	最小权限、身份校验、审计日志	不把工具调用默认视为可信动作
AI 安全误报	分级告警、人工复核、回放验证	让 AI 提高召回率，但不独占最终裁决权

表 2: Week 6 的风险与防护对应关系

5.5 拓展阅读

- Semgrep 官网: <https://semgrep.dev>

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- OWASP LLM Top 10: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- Prompt Injection 攻防综述: Simon Willison's Blog